

KAN: Kolmogorov-Arnold Networks; A Use Case Analysis

Joris Heemskerk, Henry Maathuis

February 20, 2025

Abstract

Kolmogorov-Arnold Networks (KANs) have emerged as a promising alternative to Multi-Layer Perceptrons (MLPs) by offering a more interpretable Neural Network structure. Unlike MLPs, KANs replace weight-based connections with learnable activation functions, resulting in a structured decomposition of input features. This paper provides a comparative analysis of KANs and MLPs in terms of accuracy, interpretability, data dimensionality, parameter efficiency, sample efficiency, overfitting, and catastrophic forgetting. We extend previous research by including additional interpretability-focused analyses and adding a new dataset on Sepsis-Associated Delirium (SAD). Our findings indicate that KANs offer competitive accuracy while significantly improving model transparency, making them particularly valuable in applications requiring explainability.

1 Introduction

Multi-Layer Perceptrons (MLPs) have long dominated the field of Artificial Intelligence (AI) because of their ability to approximate functions [source](#). They are widely used for predictive modeling, pattern recognition, classification tasks, and more [sources](#). MLPs consist of neurons, that receive weighted inputs and a single bias, which they sum and put through a non-linear function to provide an output [source](#). With enough layers of enough neurons, many datasets can be learned and many problems can be solved [source](#). However, MLPs are well known to be "black box" models, meaning they are not interpretable; the reasoning of the model can not be easily examined [source](#).

Some techniques like Local Interpretable Model-Agnostic Explanations (LIME) [1] and SHapley Additive exPlanations (SHAP) [2] aim to provide explanations after the model has made its decision. LIME is a technique that generates local approximations to model predictions, whilst SHAP assigns a value to each feature, indicating its contribution to a models output. However, both of these forms of post hoc analysis remain approximations. For example, using SHAP, if a single variable changes in a sample, that could affect the feature importance of all other variables. This is due to MLPs having feature interactions: when an MLP has multiple layers, different features can combine and affect each other. SHAP looks at all the possible subsets of features, meaning it looks at all their interactions, which affects each feature's individual contribution. Whilst this makes SHAP accurate, it can cause confusion when comparing two similar datapoints.

Recently, Kolmogorov-Arnold Networks (KANs) [3] have been gaining traction. They might provide the solution to the stated problem of model interpretability. KANs primarily differ from MLPs in that they learn activation functions instead of weights and biases. While MLPs train weights and biases and use a fixed activation function, KANs rely on a trainable activation function that defines the connections between neurons. This means that the relation between each input and the target variable can be visualised by a set of graphs. This makes KANs much more interpretable by design. In addition, the feature interactions of KANs and MLPs differ. KANs use linear combinations of features, which limits the complexity of interactions to explicit, predefined combinations, making the contributions more interpretable. While some degree of interaction may still emerge via the linear transformations, the explicit separation into univariate functions prevents the deep entangled interactions typical of MLPs.

This report aims to analyse and compare KANs vs MLPs in terms of accuracy, interpretability, data dimensionality, parameter efficiency, sample efficiency, overfitting, and catastrophic forgetting, in order to provided an overview of when to use KANs over MLPs. We do this by building upon research done by Poeta et al. [4], which compared MPLs and KANs on 9 different datasets,

looking at performance metrics and efficiency. We will reproduce these results using a different implementation of KANs, that allows us to also analyse the qualitative aspects surrounding interpretability. We also add a dataset on sepsis-associated delirium (SAD) introduced by Zhang et al. [5].

2 How KANs Work

Kolmogorov-Arnold Networks (KANs) are based on the Kolmogorov-Arnold representation theorem, which states that any continuous multivariate function can be decomposed into a finite composition of continuous univariate functions. This theorem provides a theoretical foundation for function approximation without relying on traditional weight-based neural networks.

2.1 Kolmogorov-Arnold Representation Theorem

The Kolmogorov-Arnold Representation Theorem (KART) asserts that any multivariate function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ can be expressed as:

$$f(x_1, x_2, \dots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^n \psi_{p,q}(x_p) \right), \quad (1)$$

where Φ_q and ψ_{pq} are continuous univariate functions that can be learned.

Instead of using layers of neurons with fixed activation functions and weighted summations, KANs directly learn these univariate functions, allowing for a more interpretable representation of the underlying function.

2.2 Spline-Based Activation Functions in KANs

KANs replace traditional weights and biases with learnable spline-based activation functions. A spline is a piecewise-defined polynomial function, typically of low degree (e.g., cubic splines), ensuring smoothness and flexibility. Formally, a spline function $S(x)$ can be expressed as:

$$S(x) = \sum_i c_i B_i(x), \quad (2)$$

where $B_i(x)$ are basis splines (B-splines) and c_i are learnable coefficients.

By using splines, KANs provide adaptive non-linearity, unlike static activation functions such as ReLU or sigmoid. Splines allow flexible and smooth transformations, adapting to data patterns. Additionally, the interpretability of KANs is enhanced since each learned function Φ_q and ψ_{pq} can be visualized, making the model more transparent. This property is particularly useful in applications where explainability is crucial. An example of such a visualisation can be found in Figure 1. Furthermore, KANs achieve efficient parameterization by optimizing a smaller set of spline coefficients instead of learning high-dimensional weight matrices, leading to improved sample efficiency.

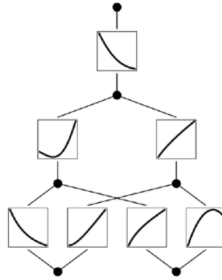


Figure 1: Example of the visualisation of a KAN model [3].

In conclusion, KANs leverage the power of the Kolmogorov-Arnold theorem by learning transformations between inputs and outputs using spline-based functional mappings, improving both interpretability and flexibility in function approximation.

3 Methodology

In order to compare KANs and MLPs, we used 9 different datasets. Eight of which are also used by Poeta et al. [4] in a study comparing the accuracy of KANs and MLPs. The ninth dataset is about sepsis-associated delirium (SAD) introduced by Zhang et al. [5]. Information on these datasets can be found in Table 3.

Dataset name	Task	Number of Features	Number Rows	Missing values	Data Type
Breast Cancer	Binary	30	569	✗	Numerical
Spam	Binary	57*	4,601	✗	Numerical
Musk	Binary	166	6,598	✗	Numerical
Dry Bean	Multiclass	16	13,611	✗	Numerical
Gamma Telescope	Binary	10	19,020	✗	Numerical
Adult	Binary	14	48,842	✓	Numerical + Categorical
Shuttle	Multiclass	7	58,000	✗	Numerical
Diabetes	Binary	21	253,680	✓	Numerical
SAD	Binary	50	14,620	✓	Numerical + Categorical

* Poeta et al. states there are 47 features, but we found their sources stating 57, as does their codebase.

3.1 Data preprocessing

The details on the preprocessing for the first 8 datasets can be found in Poeta et al. The SAD preprocessing goes as follows. We remove the following columns: *deliriumtime*, *hosp_mort*, *icu28dmort*, *stay_id*, *icustay*, *hospstay*, and *sepsistime*, leaving us with 43 features. The columns *race* and *first_careunit* are categorical, so using dummies, we convert these to separate boolean columns, leaving us with 53 feature columns. For all datasets with missing values, we drop the rows in question. The information on the data after preprocessing can be found in Table 2.

Table 2: Dataset characteristics after data preprocessing. The datasets effected in terms of shape are highlighted in bold.

Dataset name	Number of Features	Number Rows
Breast Cancer	30	569
Spam	57	4,601
Musk	166	6,598
Dry Bean	16	13,611
Gamma Telescope	10	19,020
Adult	104	45,222
Shuttle	7	58,000
Diabetes	19	253,680
SAD	53	11,196

3.2 Training Configurations

With this data, we first replicate the results from Poeta et al. The original article uses *efficient-kan*¹, which does not support pruning nor visualisations. For this reason we use *pykan*², made by the authors of the original KAN paper [3]. In order to compare accuracies between KANs and MLPs, Poeta et al. analyses their results over 5 differently seeded runs, where each run consists of training 10 models, each for 10 epochs. The 10 KAN models per run consist of one layer, with 1 through 10 nodes. The 10 MLP models per run consist of one layer with 10 through 100 nodes, in steps of 10. Each trained model is analysed on Accuracy, F1 Score, Precision, Recall, and training time.

For training, we used a HP ZBook Studio 15.6 G8 (62T51EA), with a NVIDIA A3000 GPU (6 GB VRAM) and an Intel Core i7-11850H processor. It is worth pointing out that the adult and diabetes datasets were not able to fit into GPU memory and were therefore forced to use shared memory, making them significantly slower.

¹<https://github.com/Blealtan/efficient-kan>

²<https://github.com/KindXiaoming/pykan>

4 Results

Table 3: Metrics on different models for different datasets. The results are calculated across five seeded runs, with results of each run being averaged. The time column is the total average, whilst for the other columns, the best average is displayed in the format *metric* $\pm$ *standard deviation*.

Dataset	Model	Accuracy	F1 Score	Precision	Recall	Training Time (s)
Breast Cancer	KAN	95.88 $\pm$ 0.57	95.63 $\pm$ 0.57	95.48 $\pm$ 0.59	95.82 $\pm$ 0.48	0.15
	MLP	96.32$\pm$0.23	96.08$\pm$0.24	96.09$\pm$0.28	96.08$\pm$0.28	0.00
Spam	KAN	93.37 $\pm$ 0.19	93.20 $\pm$ 0.20	93.27 $\pm$ 0.19	93.13 $\pm$ 0.21	0.23
	MLP	93.65$\pm$0.14	93.47$\pm$0.14	93.64$\pm$0.14	93.33$\pm$0.14	0.02
Musk	KAN	99.54$\pm$0.16	99.13$\pm$0.27	99.24$\pm$0.33	99.03$\pm$0.20	0.48
	MLP	95.64 $\pm$ 0.10	91.17 $\pm$ 0.25	95.31 $\pm$ 0.16	88.36 $\pm$ 0.46	0.04
Dry Bean	KAN	92.49$\pm$0.37	92.52$\pm$0.37	92.60$\pm$0.36	92.49$\pm$0.37	0.27
	MLP	92.30 $\pm$ 0.07	92.31 $\pm$ 0.08	92.37 $\pm$ 0.05	92.30 $\pm$ 0.07	0.07
Gamma Telescope	KAN	87.68$\pm$0.08	86.02$\pm$0.10	87.80$\pm$0.11	84.89$\pm$0.11	0.31
	MLP	84.65 $\pm$ 0.09	82.30 $\pm$ 0.09	84.86 $\pm$ 0.14	80.94 $\pm$ 0.08	0.09
Adult	KAN	85.85$\pm$0.06	80.17$\pm$0.09	82.61$\pm$0.11	78.51$\pm$0.09	2.13
	MLP	85.65 $\pm$ 0.03	79.83 $\pm$ 0.08	82.43 $\pm$ 0.08	78.15 $\pm$ 0.14	0.24
Shuttle	KAN	99.78$\pm$0.06	99.73$\pm$0.07	99.72$\pm$0.07	99.78$\pm$0.06	0.72
	MLP	99.23 $\pm$ 0.04	99.07 $\pm$ 0.04	99.04 $\pm$ 0.05	99.23 $\pm$ 0.04	0.28
Diabetes	KAN	86.77$\pm$0.01	58.91$\pm$0.08	72.65 $\pm$ 0.07	57.07$\pm$0.06	3.48
	MLP	86.63 $\pm$ 0.06	54.10 $\pm$ 1.11	73.86$\pm$1.70	54.14 $\pm$ 0.62	1.24
SAD	KAN	70.32 $\pm$ 0.32	68.69 $\pm$ 0.32	69.04 $\pm$ 0.35	68.51 $\pm$ 0.31	0.33
	MLP	71.54$\pm$0.20	69.19$\pm$0.49	70.58$\pm$0.04	68.78$\pm$0.37	0.05

Table 3 shows the average results of the different different configurations.

However, not all configurations provide the same level of interpretability. A KAN model with 1 layer with 100 nodes and 50 features is not practically interpretable, due to the sheer amount of activation functions (50,000). The difficult question is, at what point does it become too much. Because this goes beyond the goals of this report, we will choose the minimum configuration of 1 KAN node. This will ensure the highest interpretability, provided our tested configurations. The results are shown in Table 4.

Table 4: Average KAN results for 1 node, averaged across five seeded runs.

Dataset	Accuracy	F1 Score	Precision	Recall
Breast Cancer	93.16	92.86	92.83	93.31
Spam	94.12	93.98	93.97	93.98
Musk	97.09	94.75	93.74	96.02
Dry Bean	84.07	84.13	84.82	84.07
Gamma	86.09	84.07	86.32	82.77
Adult	85.60	79.89	82.07	78.33
Shuttle	99.36	99.21	99.12	99.36
Diabetes	86.76	58.44	72.74	56.73
SAD	72.43	70.76	71.32	70.49

4.1 Comparison to different models

The advantage of a KAN should be its interpretability, it should be more interpretable than its competition. KAN models are supposed to be a replacement for MLP models, in situations where interpretability is of importance. However, some other models might be more interpretable as MLPs as well, whilst also being more performant. In order to find out, we compared different more classical AI models with the MLP and KAN models. These results are gathered by using the default configurations provided by Python libraries scikit-learn [6] and XGBoost [7]. Each configuration was run for 10 iterations, each with a unique seed. The average results are reported in Table 5.

The table is ordered from least to most interpretable with regards to the models. MLP models are the least interpretable, as they are often considered "black boxes". While tools like SHAP and LIME can show feature importance, these are approximations and do not explain the internal

workings of the model. XGBoost and Random Forest are based on decision tree models. However, due to the large amount of these trees, it is hard to analyse the individual reasonings. XGBoost builds trees sequentially, making analysis more difficult than Random Forest. Both models can again be analysed using SHAP, but this remains an approximation. Support Vector Machines are interpretable in low-dimensional spaces, but higher dimensionality results in complex decision boundaries that are hard to visualise or explain. As stated, KAN networks are inherently interpretable. Yet, not as interpretable as Logistical Regression, which is the most interpretable in this lineup.

Table 5: Accuracy for datasets across different algorithms (ordered from least to most interpretable). The MLP results are copied from Table 3 and the KAN results from Table 4. Best results for each column are highlighted in bold.

Algorithm	Breast Cancer	Spam	Musk	Dry Bean	Gamma Telescope	Adult	Shuttle	Diabetes	SAD
MLP	96.32	93.65	95.64	92.30	84.65	85.65	99.23	86.63	71.54
XGBoost	96.05	95.39	98.95	92.76	88.18	87.05	99.83	86.53	71.71
RF	95.61	95.55	97.69	92.68	88.15	85.11	99.83	85.27	72.83
SVM	97.19	93.08	96.83	93.14	86.80	85.50	99.66	25.23	60.17
KAN (1 node)	93.16	94.12	97.09	84.07	86.09	85.60	99.36	86.76	72.43
LR	97.63	92.62	94.76	92.52	79.00	84.96	96.61	86.29	65.98

5 KAN vs MLP

In this section we compare KANs and MLPs on different metrics, using both our personal insights as well as our results as shown in section 4.

5.1 Accuracy

Accuracy is a fundamental metric for evaluating the performance of neural networks. Based on our results (Table 3) we can not conclusively show KANs being better or worse than MLPs. For most datasets the difference in accuracies between the KAN and MLP versions are minimal. The Musk dataset seem to be the exception, where KAN models outperform the MLP models by 3.9%. We hypothesise this could be caused by this dataset being separated by a more smooth function, for which KANs might be better suited. In general, the KAN model beats the MLP model in 6 out of 9 cases, with 1 clear wins and no clear losses (based on Accuracy alone). Based on these results we can conclude that KANs are generally roughly as good, if not better than MLPs, in terms of accuracy and by extension predictive power.

However, if maximal interpretability is important, we recommend using as few KAN nodes as possible. This can have an impact on the performance, both positive and negative. As shown in Table 4, 6 of the single node KAN models perform worse than the averages as shown in Table 3. We also see 3 models outperforming the average: Breast Cancer, Spam, and SAD. Most of the deviations are minor, with the exception of Dry Bean, which shows a decrease of 8.42%. We hypothesise this is because the Dry Bean dataset is multiclass, making it more difficult for a single node to separate. We do not see the same deviation for the Shuttle dataset, the only other multiclass dataset, which we hypothesise to be because this dataset is easily separable and there are fewer features.

If the maximal accuracy is desired, we conclude based on Table 5 that neither MLP nor KAN models are often the best option. Only on the Diabetes dataset does the KAN model outperform the other models. We do see that the KAN and MLP models are very close in terms of accuracy, as is the case with most of the models. Based on our results we cannot give clear advice to choose KAN over MLP models based solely on accuracy.

5.2 Training time

Training time is a critical factor in evaluating the efficiency of neural networks, especially in resource-constrained environments. Our results conclude that KANs are significantly slower to train. As shown in Table 3, KANs are on average a factor of 4 slower to train compared to MLPs. Results in Poeta et al. showed KANs to be slower on average by a factor of 2.1, because of their

use of efficient-kan over our use of pykan. When using KANs for their predictive performance alone, using efficient-kan is recommended. This library improves training times significantly, but removes the interpretability features. If interpretability is most important, we recommend using pykan instead.

5.3 Interpretability

Interpretability refers to the ability to understand and explain how a model makes its predictions. Highly interpretable models allow researchers and practitioners to analyse individual components, diagnose errors, and build trust in their decisions. In machine learning, interpretability is especially important in fields such as healthcare, finance, and law, where understanding model reasoning is crucial.

KANs offer a higher degree of interpretability compared to MLPs due to their architecture. Unlike MLPs, which rely on learned weight matrices and non-linear activations, KANs explicitly model relationships using kernel functions. Each node in a KAN applies a learnable transformation that can be visualized as a function mapping, making it possible to directly observe how input features influence outputs (see Figure 1 for an example). This transparency allows for a more intuitive understanding of feature interactions, as opposed to the “black box” nature of MLPs, where feature effects are entangled within weight matrices.

Despite this advantage, KANs do not achieve the level of interpretability seen in simpler models such as Linear Regression. In linear models, feature contributions are explicitly represented by coefficients, making it easy to quantify their impact on predictions. A weight of 2.5 for a particular feature, for example, means that increasing the feature by one unit increases the prediction by 2.5 units. KANs, on the other hand, introduce non-linearity through kernel transformations, making feature contributions more complex to analyze. While KANs provide visualization of learned functions, they do not offer the same straightforward numerical interpretation as linear models.

Moreover, interpretability in KANs decreases as dimensionality increases. A single KAN node with one feature is highly interpretable as it can be visualized as a simple function graph. However, as the number of features grows, so does the complexity of interpreting the model. With 1,000 features, for example, interpretability becomes impractical, as it would require analyzing 1,000 separate graphs. While each function is still individually accessible, the volume of information makes it more difficult to interpret fully.

Based on this reasoning, when requiring interpretability, we would recommend starting out with simple linear models. As seen in Table 5, Linear Regression outperformed KANs in 2 of the 9 datasets (Breast Cancer and Dry Bean) and got comparable results in 3 (Spam, Adult, and Diabetes). If it turns out that Linear models do not yield the required performance, we would recommend using KANs, because of their ability to work with non-linearly separable datasets. If KANs are used, we recommend a less is more approach where the less KAN nodes and features used, the more interpretable the model. The pruning of KAN models can also positively impact interpretability, by removing features and therefore information to interpret, with minimal cost to performance.

5.4 Data dimensionality

Data dimensionality refers to the number of features in a dataset. Low-dimensional datasets have fewer features, while high-dimensional datasets can have hundreds or even thousands of features. High-dimensional data poses challenges such as increased computational cost, risk of overfitting, and difficulties in interpretability. The datasets we used range from 7 to 166 features (see Table 2). We will label Adult (104) and Musk (166) as high-dimensional datasets and Shuttle (7) and Gamma Telescope (10) as low-dimensional for this analysis.

Based on our results from Table 3 we conclude that there is no sufficient basis to conclude either models as better in low- or high-dimensional datasets, in terms of performance. However, we do see training times being comparatively slower for KANs in high-dimensional datasets. On high-dimensional datasets, KANs were on average a factor of 9.3 slower to train, whilst low-dimensional datasets they were only a factor of 2.8 slower. This means higher-dimensionality has more of a negative affect on training KANs compared to MLPs.

In addition, we can differentiate in terms of interpretability. We know that interpretability decreases when dimensionality increases. Each added feature likely adds multiple connections to the KANs, each adding to the pending interpretation. This means that low-dimensional datasets are preferable when using KANs.

5.5 Parameter efficiency

Parameter efficiency refers to the ability of a model to achieve high performance with a relatively small number of parameters. A model that requires fewer parameters to achieve the same or better accuracy is considered more efficient, as it reduces memory usage, speeds up inference, and lowers the risk of overfitting. When comparing, KANs often demonstrate greater parameter efficiency to MLPs, due to their fundamental design differences.

MLPs rely on densely connected layers, where each neuron in a layer is connected to every neuron in the subsequent layer. This design leads to a rapid increase in the number of parameters as the network scales. A deep MLP with thousands of neurons per layer can easily require millions of parameters. In contrast, KANs replace traditional activation functions with learnable univariate and bivariate functions, which allows them to represent complex relationships without the need for excessive weight matrices. Instead of relying on depth and width to model intricate functions, KANs leverage these learned function transformations, drastically reducing the total number of parameters required, making them more efficient.

5.6 Sample efficiency

Sample efficiency refers to a model’s ability to achieve high performance with a limited amount of training data. A sample-efficient model can generalize well from fewer examples, reducing the need for large datasets and extensive training. When comparing, KANs have the potential to be more sample efficient over MLPs, due to their fundamentally different approach to function approximation.

MLPs rely on learning complex mappings through layers of weighted connections and nonlinear activations. This process often requires a vast amount of data to generalize effectively, as the model must infer high-dimensional relationships through iterative optimization. Small datasets may cause MLPs to struggle with overfitting because their large number of parameters can memorize the training set rather than learning generalizable patterns. Regularization techniques, such as dropout and weight decay, are commonly used to mitigate this issue, but they do not inherently resolve the sample inefficiency of MLPs.

KANs, on the other hand, learn functions in a structured and interpretable manner in the splines. This allows KANs to extract meaningful functional relationships with fewer training examples, leading to improved generalization even in low-data regimes. Even with a small number of training examples, KAN models will still learn a complete mapping for each data point using the splines, resulting in a sensible (though not always accurate) generalised inference for new data.

However, the degree to which KANs are more sample efficient than MLPs depends on the specific task and dataset. If the underlying function mapping is highly complex and requires intricate feature interactions, KANs may still need a significant amount of data to approximate the relationships effectively. Furthermore, in high-dimensional spaces, where interactions between features are intricate and non-trivial, KANs may not always be able to generalize with significantly fewer samples compared to well-regularized MLPs.

5.7 Overfitting

Overfitting occurs when a model learns patterns that are too specific to the training data, failing to generalize well to unseen examples. This typically happens when a model has too many parameters relative to the amount of available training data, allowing it to memorize noise rather than capturing meaningful relationships. When comparing KANs and MLPs, the susceptibility to overfitting depends on their respective architectures and learning strategies.

MLPs, especially deep ones with millions of parameters, are highly prone to overfitting when training data is limited. Since MLPs learn mappings through weight-based transformations, they

often require regularization techniques such as dropout, weight decay, and batch normalization to mitigate overfitting. If these techniques are not carefully tuned, an MLP can easily memorize training examples rather than learning generalizable patterns, leading to poor performance on test data.

KANs, in contrast, have a different learning mechanism that can make them both more and less susceptible to overfitting, depending on the scenario. KANs use smooth function approximations, which can inherently regularize learning by promoting structured transformations rather than arbitrary weight-based adjustments. This means that a well-designed KAN may be less likely to overfit compared to an equivalently sized MLP because it captures functional relationships in a more constrained and interpretable manner.

However, KANs are not immune to overfitting, particularly when dealing with high-dimensional data. As the number of input features increases, the complexity of the function space KANs need to model also grows. If each input variable is assigned its own univariate transformation, a KAN may end up modeling fine-grained variations in the training data rather than generalizing well across different samples. This can lead to overfitting in a manner similar to MLPs, particularly when the dataset is small relative to the model's expressive power.

Another factor influencing overfitting in KANs is their reliance on predefined basis functions. If a KAN is not properly regularized, or if the chosen basis functions allow too much flexibility, the network may fit noise. However, if the splines do not allow for enough flexibility, the integrated patterns of the data might not be learnable. So whilst KANs are less likely to overfit, they might be more susceptible to underfit. Proper tuning of their flexibility is essential to balance between generalization and expressivity.

5.8 Catastrophic forgetting

- Are KANs more or less susceptible to catastrophic forgetting?

6 Conclusion

- This section is new, and added because of the high amount of information in section 5.
- This section should contain a simple reference for when we recommend using KANs and when we do not.

7 future of KANs

- Can we use KANs for other types of data? Images, audio, language, tabular data, sequential data, etc?
- Has there been research into using KANs as cornerstones for Transformer models?
- Are there ways to optimize KANs for GPUs?

References

- [1] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?": Explaining the predictions of any classifier, 2016. URL <https://arxiv.org/abs/1602.04938>.
- [2] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 4765–4774. Curran Associates, Inc., 2017. URL <http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf>.

- [3] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. Kan: Kolmogorov-arnold networks, 2024. URL <https://arxiv.org/abs/2404.19756>.
- [4] Eleonora Poeta, Flavio Giobergia, Eliana Pastor, Tania Cerquitelli, and Elena Baralis. A benchmarking study of kolmogorov-arnold networks on tabular data, 2024. URL <https://arxiv.org/abs/2406.14529>.
- [5] Yang Zhang, Juanjuan Hu, Tianfeng Hua, Jin Zhang, Zhongheng Zhang, and Min Yang. Development of a machine learning-based prediction model for sepsis-associated delirium in the intensive care unit. *Scientific Reports*, 13(1):12697, 2023.
- [6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [7] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pages 785–794, New York, NY, USA, 2016. ACM. ISBN 978-1-4503-4232-2. doi: 10.1145/2939672.2939785. URL <http://doi.acm.org/10.1145/2939672.2939785>.